TASK 2 – Choose the Data Types 4 Marks

Bloom's Level: K4 – Analyse Select a C data type for every data item identified in Task 1.

Discuss:

• The type chosen (char, short, int, long, long long, unsigned, float, double) and its size using sizeof on your system

• The range of each type using limits.h and float.h, and whether it is sufficient for a district

• Why money should not be stored as float rupees, and the alternative of storing paise in an integer

• What happens to a 16-bit unsigned meter register after 65535 Show one example where float gives a wrong total when 4,000 bills are added, and how your choice avoids it.

Complete the Code Complete TODO 1 below without using if–else, and execute the program.

```
#include <stdio.h>

#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */ typedef
unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)

{

    return (meter_t)(current - previous);

}

int main(void)

{

  printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n", sizeof(meter_t),
(unsigned)USHRT_MAX);

  printf("Consumption: %u units\n", consumption(1200, 1530));

  printf("Consumption: %u units\n", consumption(65400, 120)); /* rolled over */

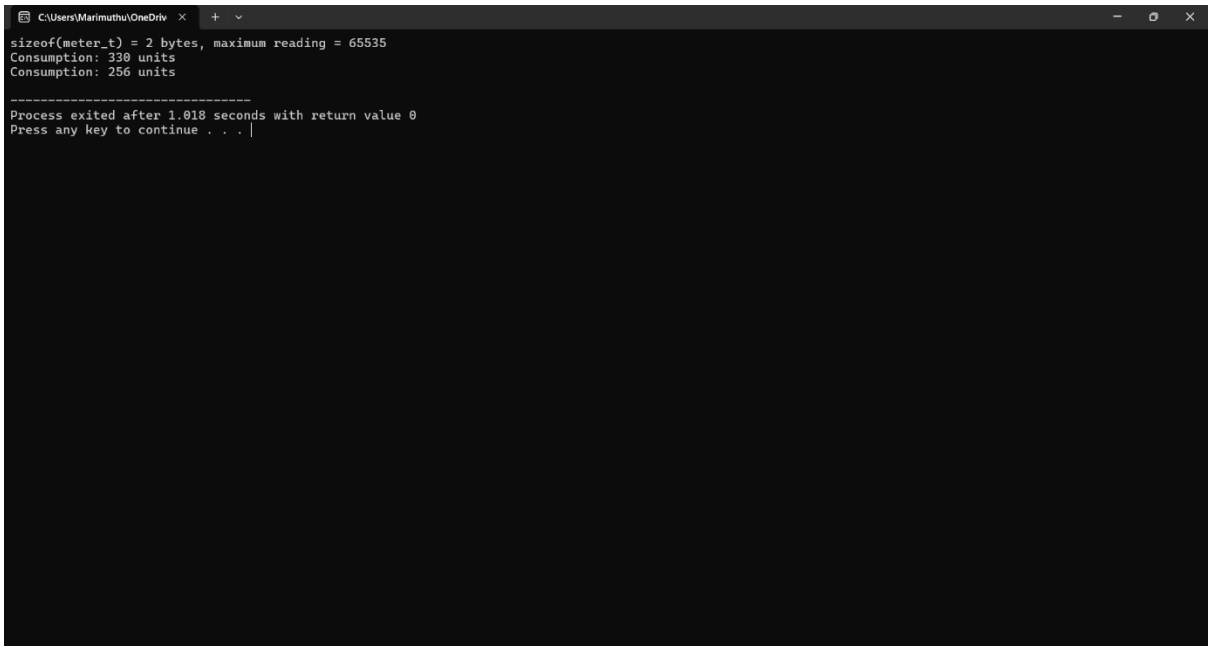
return 0;
```

}

Expected output: sizeof(meter_t) = 2 bytes, maximum reading = 65535

Consumption: 330 units

Consumption: 256 units



Explain why the second result is 256 and not a negative number.

Alternative Concept and Complexity Analysis

Suppose money were stored as double rupees instead of long paise. What would the billing solution be, and where would rounding be applied? Analyse the time per addition (O(1) in both) and the space per value, and compare how the rounding error grows when n bills are added. Evidence Expected A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

1. Data type selection

Data item	Suitable C data type	Reason
Previous meter reading	meter_t (unsigned short)	Stores a non-negative meter reading from 0 to 65535 on the specified 16-bit register.
Current meter reading	meter_t (unsigned short)	Stores the current register value.

Household number	int or long long	Stores the household identifier, depending on its maximum size.
Category	char or character array	Stores a category code or category name.
Units consumed	unsigned int	Stores the calculated consumption, including consumption across a meter rollover.
Energy charge	Integer paise, using a suitable integer type	Avoids binary floating-point rounding errors in money calculations.
Fixed charge	Integer paise, using a suitable integer type	Stores the fixed charge in whole paise.
Total bill	Integer paise, using a suitable integer type	Adds the bill components without floating-point rounding.

Sizes and ranges of C data types

The exact sizes depend on the compiler and system. Use sizeof, limits.h, and float.h to obtain the values on your own computer.

Type	**How to check its limits**	**Typical size**
char	CHAR_MIN, CHAR_MAX	1 byte
short	SHRT_MIN, SHRT_MAX	2 bytes
unsigned short	USHRT_MAX	2 bytes
int	INT_MIN, INT_MAX	4 bytes
long	LONG_MIN, LONG_MAX	4 or 8 bytes
long long	LLONG_MIN, LLONG_MAX	8 bytes
unsigned int	UINT_MAX	Usually 4 bytes
float	FLT_MIN, FLT_MAX, FLT_DIG	Usually 4 bytes
double	DBL_MIN, DBL_MAX, DBL_DIG	Usually 8 bytes

These are typical sizes, not guarantees for every C implementation. The program below prints the actual values for your compiler.

3. Why money should not be stored as float rupees

A float cannot represent every decimal fraction exactly. Repeated additions may accumulate small rounding errors.

For example, adding ₹0.10 to a float total 4,000 times may produce a result slightly different from ₹400 on common systems.

Better approach: Store money as integer paise.

- ₹0.10 = 10 paise
- 4,000 bills × 10 paise = 40,000 paise
- 40,000 paise = ₹400.00

Integer addition avoids floating-point rounding errors, provided the integer type does not overflow.

4. What happens when a 16-bit unsigned meter register exceeds 65535?

A 16-bit unsigned register has values from 0 to 65535. After 65535, it rolls over to 0.

For the given example:

- Previous reading = 65400
- Current reading = 120
- Register capacity = 65536

Consumption is:

(65536-65400)+120=256

The answer is 256 units, not a negative number, because unsigned 16-bit arithmetic wraps around modulo 65536.

6. Alternative concept: Store money as double rupees

If money were stored as double instead of integer paise:

- Each bill would be calculated in rupees using double.
- The program would round the amount to two decimal places at the appropriate billing stage.

- Repeated additions could accumulate small floating-point errors.
- Rounding each bill before summation can differ from summing unrounded values and rounding the final total.

Comparison	Integer paise	double rupees
Addition time	O(1)	O(1)
Space per value	Fixed; depends on integer type	Typically 8 bytes
Precision	Exact for whole paise, within range	Decimal fractions may be approximate
Rounding	Not required for whole-paise additions	Required when applying currency rules

Both approaches use constant time per addition. Integer paise is generally preferable for ordinary billing when charges are expressed in whole paise and the chosen integer type has sufficient range.

7. Experimental evidence to include

Add these items to your Task 02 report:

1. The data type table with actual sizes and ranges from your compiler.
2. The complete program and screenshot of its output.
3. The rollover calculation showing why consumption is 256 units.
4. A short comparison of float and integer paise, including the 4,000-bill example.

Code :

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
```

```
    printf("char: %zu byte, range %d to %d\n",
        sizeof(char), CHAR_MIN, CHAR_MAX);

    printf("short: %zu bytes, range %d to %d\n",
        sizeof(short), SHRT_MIN, SHRT_MAX);

    printf("unsigned short: %zu bytes, max %u\n",
        sizeof(unsigned short), USHRT_MAX);

    printf("int: %zu bytes, range %d to %d\n",
        sizeof(int), INT_MIN, INT_MAX);

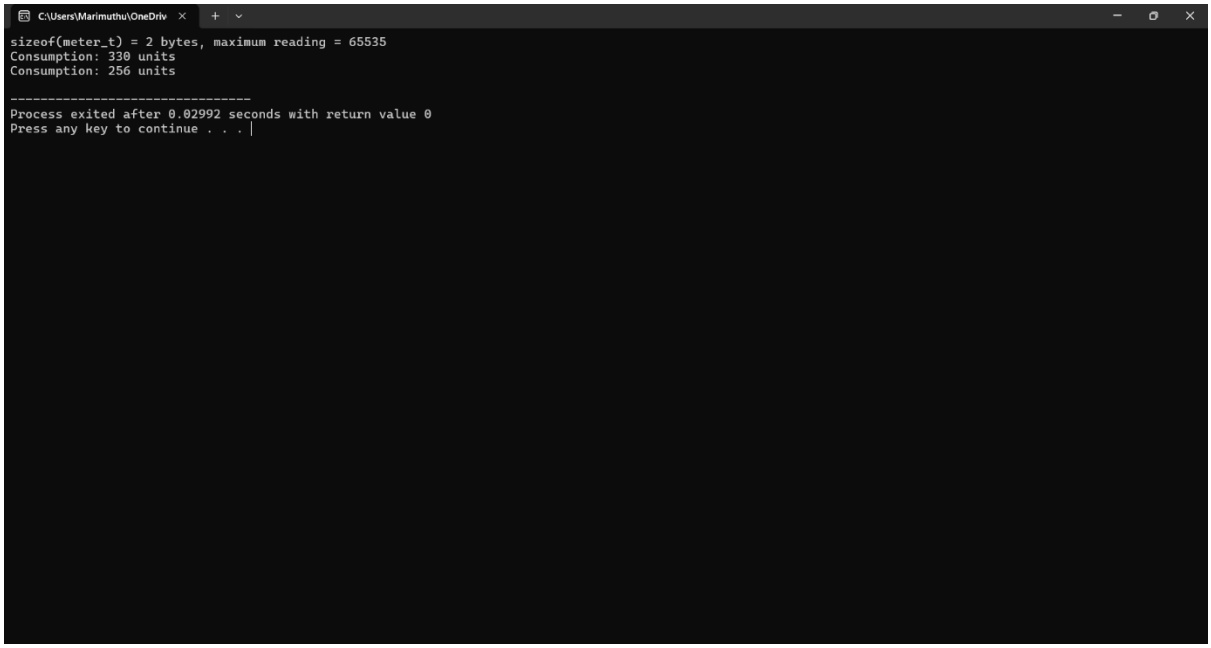
    printf("long: %zu bytes, range %ld to %ld\n",
        sizeof(long), LONG_MIN, LONG_MAX);

    printf("long long: %zu bytes, range %lld to %lld\n",
        sizeof(long long), LLONG_MIN, LLONG_MAX);

    printf("unsigned int max: %u\n", UINT_MAX);
    printf("float: %zu bytes, decimal digits %d\n",
        sizeof(float), FLT_DIG);
    printf("double: %zu bytes, decimal digits %d\n",
        sizeof(double), DBL_DIG);

    return 0;
}
```

Output:



Conclusion: The selected data types must handle the full range of meter readings, household records and billing totals. Unsigned arithmetic correctly handles the specified meter rollover, while integer paise avoids floating-point rounding errors in currency calculations.